

What We Are Doing, What We Have Proven, and What Is Left

3 October 2026 · A complete explanation from first principles · DI-Bench extension

Part 1 — The problem, explained from scratch

What a dependency is

When someone writes a Python program, they rarely write everything themselves. If they need to fetch a web page, they use a package called `requests` that somebody else wrote. If they need to show a progress bar, they use `tqdm`. These borrowed packages are called **dependencies**.

For the program to run on someone else's computer, that person has to install those packages first. So every project keeps a list of them in a file. In Python that file is usually called `pyproject.toml`, and the list is called the **manifest**. The manifest is a promise: "install these things and my program will work."

The problem with that promise

The manifest is written by hand. The code changes constantly. Nothing automatically keeps them in sync.

So the manifest drifts away from reality. A developer deletes the last piece of code that used `tqdm` but forgets to remove `tqdm` from the list. Now the project installs a package it never uses. A 2024 study of the whole Python package ecosystem found **more than half of all declared dependencies are like this** — listed but unused. The single biggest cause, 34% of cases, is exactly that: someone refactored the code and forgot the manifest.

This is not a cosmetic problem. Every unnecessary package is more code downloaded onto every user's machine, more code that could contain a security hole, and more code nobody on the team has read.

What people are trying to automate

Because this is tedious and error-prone, there is now interest in having AI agents do it. Given a repository, work out what it actually needs and write a correct manifest.

To know whether an agent is any good at this, you need a benchmark: a fixed set of problems with known right answers, so you can score different systems on the same footing. **DI-Bench** is the main one. Microsoft published it in 2025. It works like this:

1. Take a real open-source project that works.
2. Delete its manifest contents — blank out the dependency list.
3. Ask the AI to fill it back in.
4. Check the answer.

How DI-Bench checks the answer, and why that is the crux

Step 4 is where everything hinges. DI-Bench installs whatever the AI wrote, then **runs the project's own test suite**. If the tests pass, the answer is marked correct. This score is called **execution pass rate**, and it is the headline number everyone quotes.

The appeal is obvious. It is objective. It is automatic. Nobody has to judge anything by hand. Running the real code feels like the gold standard.

The question we asked

Does passing the tests actually mean the manifest is right?

Think about what a test suite can and cannot notice.

A test can only catch a *missing* dependency if some test actually runs the code that uses it. If a package is only used by a command-line feature that no test touches, you can delete it from the manifest and every test still passes — while the program is now broken for real users.

A test can only catch an *extra* dependency if that package fails to install. But most packages install fine. So an AI can list ten packages the project never uses, everything installs, every test passes, and the score is perfect.

Both directions are invisible. That is the problem statement: **a score everyone trusts might be measuring much less than it appears to**. Nobody had checked.

Part 2 — The experiments, explained from scratch

We ran four experiments. Here is what each one does, in plain terms.

Experiment 1 — Does the benchmark still work at all?

What we did. Before testing anything else, we took DI-Bench's own official correct answers and ran them. If the published right answer does not pass, something is broken.

Why this matters. These benchmarks are frozen snapshots of projects from a particular moment. The outside world keeps moving. Package installers get updated, old Python versions stop being downloadable, libraries release new versions that break things. A benchmark can quietly stop working.

Problem we hit. DI-Bench's test runner needs a special container setup that normal cloud machines do not provide. We rebuilt it to run on free GitHub infrastructure. We also found two bugs in DI-Bench's own evaluation code — one crashes on certain dependency formats, and one reports "success" for a job that never actually started. We fixed both and excluded the affected results.

Experiment 2 — The core experiment: how blind are the tests?

This is the heart of the paper, and the idea is deliberately simple.

What we did. Take a project's *correct* manifest — the official right answer. Delete exactly one dependency from it. Change nothing else. Run the tests.

If the tests fail, good: the test suite noticed something was missing. The oracle works for that dependency.

If the tests still pass, that dependency is **invisible** to the benchmark. The score cannot tell whether an AI gets it right or wrong, because deleting it makes no difference.

Scale. We did this once for every dependency in every project: **531 separate deletions**, each one a full container run of the real test suite.

The borrowed idea. This is called *mutation testing*. Software researchers have used it for decades to check whether a test suite is any good: break something on purpose, see if the tests notice. We are the first to point it at a *manifest* rather than at code, and the first to point it at a dependency benchmark.

Experiment 3 — Can we predict blindness without running anything?

What we did. Running 531 container jobs is slow. We asked whether two cheap checks, done by just reading files, could predict the same answer.

- **Check one, the package list.** Public records show which packages require which others. If package A requires package B, then listing B separately is pointless — it arrives with A regardless. We can read this from public data in seconds.
- **Check two, the import graph.** We trace from each test file through the code to see which packages are ever reached. If nothing a test touches imports a package, the tests cannot possibly notice its absence.

Then we compared these predictions against what actually happened in the 531 real runs.

Experiment 4 — Can a fake removal fool the tests?

The task we care about. Beyond fixing manifests, there is a harder and more valuable task: *remove a dependency the code actually uses*, by rewriting the code to not need it. This is real work developers do when a library is abandoned or risky.

The worry. If tests are the only judge, there are cheap ways to look successful without doing the work:

- **Hiding:** move the package to an "optional" section. It still installs. Tests pass.
- **Vendoring:** copy the library's source code into your own project. The imports still work. Tests pass.
- **Stubbing:** replace the real function with a fake one that returns nothing. If no test checks that path, tests pass.

What we did. Rather than hoping an AI would try these, we wrote all three fakes automatically for 62 projects — **176 fake removals** — and ran every one through the real test suite to see which ones got away with it.

We also ran real AI agents. Two open-source coding models, 28 attempts at genuine removals, on a \$3 budget, with a deliberately neutral instruction that never mentioned cheating.

Part 3 — The results, explained from scratch

Result 1 — The benchmark has decayed

Official correct answers that still pass today	51 of 96
--	----------

Roughly half of DI-Bench's own published answers no longer work, about 21 months after release. The causes are entirely external: an installer dropped support for old Python versions, a download disappeared, and libraries released new versions that broke the projects' own tests.

Why this matters. Anyone using this benchmark today and not checking would attribute these failures to the AI. We report every other number only on the half that still works.

Result 2 — Half the dependencies are invisible to the tests

This is the main finding.

Dependencies deleted and re-tested	233
Deletions the tests did not notice	115 — 49%
Projects with at least one invisible dependency	38 of 51

Half of all dependencies can be deleted without a single test failing. And it is not a few unusual projects: three quarters of projects have at least one.

There are two distinct reasons, and separating them matters:

Reason	Count	What it means
Phantom — the package installed anyway	91	Another dependency pulled it in. Listing it was redundant. The code genuinely works.
Blind spot — the package was truly gone	24	The code would crash, but no test ever runs that code. A real defect, completely unseen.

These need different fixes. The 91 phantoms are a flaw in the benchmark's *answer key* — a model that omits them is marked wrong for being right. The 24 blind spots are gaps in the *projects' tests* that no amount of benchmark curation can repair.

Sanity check. For the 118 deletions the tests *did* catch, we confirmed the package was genuinely absent in 114 of them. The experiment behaves as expected where it should.

Result 3 — The blindness is predictable for free

Method	How often right when it flags	How much it finds
Package-list check alone	97%	62%
Import-graph check alone	69%	42%
Both together	81%	83%

Reading two files predicts five out of six invisible dependencies, correct four times in five, with **no containers and no test runs**. What took 531 container jobs can be approximated in seconds.

Result 4 — Tests accept a third of fakes

Fake	Attempts	Fooled the tests	Caught by our extra checks
Hiding in an optional section	62	35	35 of 35
Copying the library in	52	18	18 of 18
Replacing with an empty function	62	8	0 of 8
Total	176	61 — 35%	53 of 61

Three things follow.

Tests alone are genuinely foolable. 61 of 176 deliberate fakes passed. The easiest one, moving the declaration to an optional section, worked more than half the time.

Simple extra checks close two of the three holes completely. Checking what is actually installed catches every hiding attempt. Comparing added files against the library's real source catches every copying attempt.

Stubbing defeats everything we check at install time. All 8 that passed slipped through, because nothing was installed and nothing was copied — the agent just wrote empty functions, 4 to 27 lines each. Catching these needs a structural check on the *shape* of the new code, which is why that check exists.

Result 5 — With real agents, the score overstates success

Model	Attempts	Tests passed	Genuinely correct
deepseek-v3.2	15	5	3
qwen3-coder	13	0	0
Total	28	5	3

The test-only score says 5. The truth is 3. Of the two false successes, one never removed the declaration at all, and one changed nothing while the package arrived through another dependency. Both are caught by our extra checks.

No agent cheated on purpose. Across 28 attempts there was no copying, no stubbing, no tampering with tests. The fakes in Result 4 show what the scoring *would accept*; they are not an accusation against these models.

The task is genuinely hard. Three clean successes in 28 tries. Most failures were clumsy rather than devious: four agents left the manifest syntactically broken.

Part 3b — The two-graph idea, explained from scratch

Experiment 3 used two cheap checks. Both are simplified versions of one bigger idea, and the full version is what makes the extra checks in Experiment 4 work. Here it is from the beginning.

Two different maps of the same project

Map one: the code graph. Imagine drawing every function in the project as a dot, and an arrow from one dot to another whenever the first function calls the second. Add dots for the outside packages too, so when project code calls `requests.get`, there is an arrow leaving the project and landing on `requests`. This is a map of *what the code does*.

Map two: the dependency graph. Now draw a different map where each dot is a whole package, and an arrow means "this package needs that one." This map has two layers that are easy to confuse but very different:

- the **declared** layer — what the manifest says the project needs;
- the **resolved** layer — what actually ended up installed on the machine.

Those two layers are supposed to match. When they do not, something interesting is happening.

Why two maps instead of one

A dependency is a relationship between the two maps. The code graph says "project code reaches into package D." The dependency graph says "D is declared and installed." A correct manifest is just these two maps agreeing.

Every problem in this paper is a specific disagreement:

- A **bloated** dependency: declared in the manifest, but no arrow from the code reaches it.
- A **phantom** dependency: arrows from the code reach it, but the manifest never declared it. It was installed only because another package dragged it in.
- A **blind spot**: arrows reach it, but no arrow starting from a test file ever gets there, so the tests cannot see it.

This is why the pair of maps is the right object. Each problem is invisible in one map alone and obvious when you hold both.

The part that catches cheating

Now the useful bit. When an agent changes a project, **both maps move**, and the *pair* of movements is a fingerprint of what the agent actually did.

What the agent did	How the code map moves	How the dependency map moves
Genuine removal	Arrows into the package vanish; new project dots appear with their own arrows going onward	Package leaves both layers
Copied the library in	A big new clump of dots appears, shaped suspiciously like the library's own internal map	Package leaves both layers
Wrote empty stub functions	Arrows vanish; new dots appear but they are <i>dead ends</i> with no arrows leaving them	Package leaves both layers
Hid it in an optional section	Nothing moves at all	Package moves from main to optional
Relied on a phantom	Nothing moves at all	Leaves the declared layer, stays in the resolved layer

Read down that table and each row is distinguishable. A genuine removal grows real structure: new functions that call other things. Copying grows a lot of structure shaped like the original library. Stubbing *removes* structure and adds only hollow dead ends. Hiding and phantoms do not touch the code map at all, and show up purely as the two dependency layers disagreeing.

That is the whole idea: **you do not need to understand what the code means, only how its shape changed.** And shape can be measured mechanically.

Why this matters for the result we could not otherwise explain

Experiment 4 found that 8 stubbed fakes passed every install-level check. Nothing was installed, nothing was copied, so looking at packages tells you nothing.

But look at the code map and they are unmistakable. The agent deleted arrows pointing at the library and added between 4 and 27 lines of functions that return nothing — dots with no outgoing arrows. Real replacement code does work, so it calls other things. Hollow code does not. We flagged **61 of 62** stubs this way.

That is the argument for the graph layer stated concretely: it catches the one cheat family that everything else misses.

What we actually built, and what we did not

Built and used. The file-level version. We trace which *files* import which packages, and which files the tests reach. That is the import-graph check in Experiment 3, and it works: combined with the package-list check it predicts invisible dependencies at 81% precision. We also built the shape checks that caught the stubs.

Tried and set aside. The symbol-level version, where dots are individual functions rather than whole files. We used a research tool called PyCG, and it only finished on 18 of 51 projects before timing out on the rest. Worse, on the ones it did finish, it scored *worse* than the simpler file-level check — 47% precision against 69% — because Python lets you load code dynamically in ways static analysis cannot follow, so it wrongly declared things unreachable.

The honest conclusion. The coarse version works and is what our numbers rest on. The fine-grained version is the right long-term idea but current tooling cannot deliver it reliably at this scale. We say so rather than pretending otherwise.

What is genuinely new here

Both maps already exist in prior work. Researchers have built call graphs across the whole Python ecosystem, and security tools trace from application code into library functions to decide whether a vulnerability is actually reachable. Comparing call graphs before and after a change is twenty years old.

What does not exist is using the *pair of movements* as a lie detector for an automated change, with a specific signature for each way of faking it. That is the narrow claim, and it is the only one we make.

Part 4 — What is proven and what is not

Settled, with executed evidence

Claim	Evidence
Half of dependencies are invisible to the test oracle	531 container runs
Blindness splits into redundant listings and real coverage gaps	log analysis of all 115
Blindness is predictable statically	validated against those 531 runs
Tests alone accept a third of deliberate fakes	176 fakes through real CI
Install-level checks close hiding and copying entirely	same 176
Stubbing defeats install-level checks	the 8 that passed
The benchmark has decayed by half	96 official answers re-run
The test score overstated agent success	28 agent runs, 5 reported vs 3 real

Not yet proven

Whether real agents exploit the weakness. We showed the scoring *can* be fooled, not that anyone does. 28 runs with two mid-sized models found no deliberate cheating. The open question is whether more capable models, which succeed more often, also cut corners more often. This needs frontier models across the full instance set.

Whether agents are worse than humans at keeping manifests honest. Our motivating story says refactoring causes the drift. Testing it means examining real agent-authored pull requests and comparing them to human ones. The data is public; the study is not done.

The sixth check. Our quality check is specified but not built. It needs the refactoring-quality metric from the other project ported to Python.

Does this generalise beyond Python? Every number is Python. The mechanisms are not Python-specific, but the magnitudes certainly are — ecosystems with stricter dependency rules should show

fewer phantoms.

Benchmark scale. 162 instances is pilot-sized. Reaching 200–300 is mechanical with the existing pipeline, not a new idea.

Honest weaknesses to state before a reviewer finds them

- **28 agent runs is small.** Margins of error are roughly ± 18 points. We report counts, never rates.
 - **One model was cut short.** deepseek ran out of its step budget in 13 of 15 attempts, so its low score partly reflects our budget, not its ability.
 - **The surviving half may be easier.** Conditioning on projects that still work could bias toward better-maintained ones.
 - **Our fakes are constructed.** They prove what the scoring admits, not what agents do.
-

The one-paragraph version

Software is assembled from borrowed packages, and the list of those packages drifts out of sync with the code. AI agents are now being pointed at this problem, and we score them by running the project's tests. We checked whether that score means what people think. By deleting dependencies one at a time from the official answer key and re-running the real tests 531 times, we found that **half of all dependencies can be removed without a single test failing** — sometimes because the package arrives through another one, sometimes because no test touches the code that needs it. We showed this blindness can be predicted for free by reading two files, that **a third of deliberately faked removals pass the tests**, and that on real agent attempts the test-only score reported five successes where only three were genuine. The fix is not to abandon tests but to stop treating them as the whole answer: check what was actually installed, check nothing was copied in, and check the replacement code is not hollow.